

Utilización avanzada de clases III. Interfaces

Esteban Álvarez

Interfaces

- Una interfaz permite dar una **funcionalidad** o **comportamiento específico** a una clase
- **No confundir** con interfaz gráfica de usuario (GUI)
- Tenemos un ejemplo claro en la Unidad Didáctica de Ficheros
 - Para almacenar un objeto en un fichero, este debía ser **Serializable**

```
public class Profesor extends Persona implements Serializable{
```

- Que la clase Profesor sea Serializable significa **que se puede serializar**, es decir, guardar en un fichero
- Por tanto, se ha añadido una nueva funcionalidad a esa clase

Interfaces

- Otro ejemplo es el de la ordenación de objetos en una colección usando el método `Collections.sort()`
- [Presentación de la Unidad 5](#)
- Persona implementa la interfaz **Comparable** y a partir de ahí, ya se puede comparar

```
public class Persona implements Comparable<Persona> {
```

- Al implementar Comparable estamos añadiendo una nueva funcionalidad a la clase y nos **obliga a implementar** el método `compareTo`

```
    public int compareTo(Persona otra) {  
        if (this.altura < otra.altura) {return -1;}  
        if (this.altura > otra.altura) {return 1;}  
        return 0;}  
    }
```

Interfaces

- Una interfaz es una **clase abstracta pura**
 - Si tienen algún atributo, es **constante** (final)
 - **Todos** sus métodos son **abstractos** (no tienen código)
- Las clases que **implementen** la interfaz deben escribir los métodos
- De esta manera, la clase **adquiere el comportamiento** de la interfaz
- **IMPORTANTE:** No se trata de una herencia si no de una **implementación de comportamientos**

Interfaces

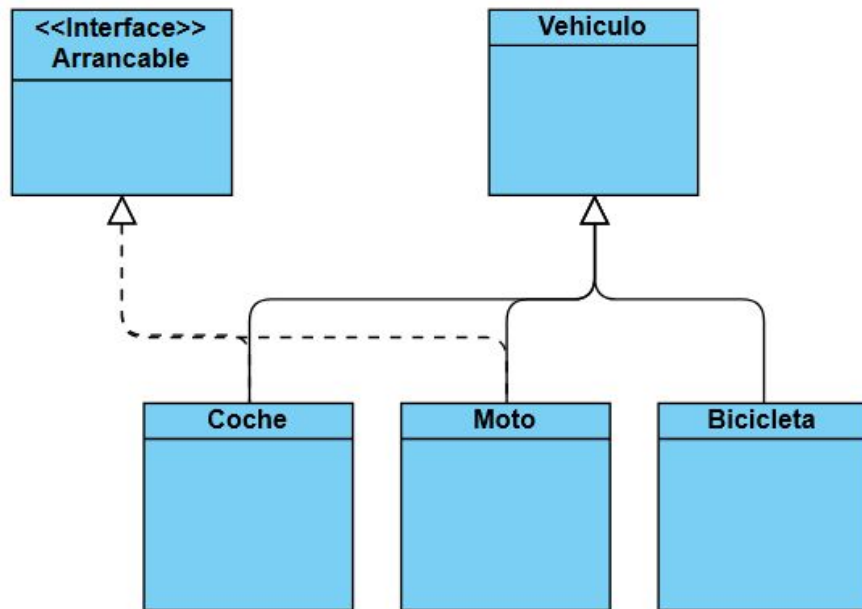
- Los métodos sin implementar de una interfaz **indican un comportamiento**
- Pero **no especifican** cómo será ese comportamiento (no tienen código!)
- El **comportamiento exacto dependerá** de las características de la clase que implemente la interfaz

Interfaces

- Ejemplo: Tenemos las clases Bicicleta, Coche y Moto y la interfaz Arrancable (que se puede arrancar)
 - No tiene sentido que la clase Bicicleta implemente la interfaz Arrancable porque una bici no se puede arrancar
 - Sin embargo un Coche y una Moto sí se pueden arrancar
 - **public class** Coche **implements** Arrancable{
 - Para arrancar un Coche, hay que meter la llave en el contacto y girar una llave (vamos al método clásico)
 - Para arrancar una moto, hay que dar al contacto y pulsar un botón
 - Coche y Moto se pueden arrancar pero de formas diferentes
- Si Bicicleta, Coche y Moto heredan de Vehículo:
 - Vehículo **no puede implementar** la interfaz Arrancable porque bicicleta no se puede arrancar

Interfaces

- Esquema del ejemplo anterior:

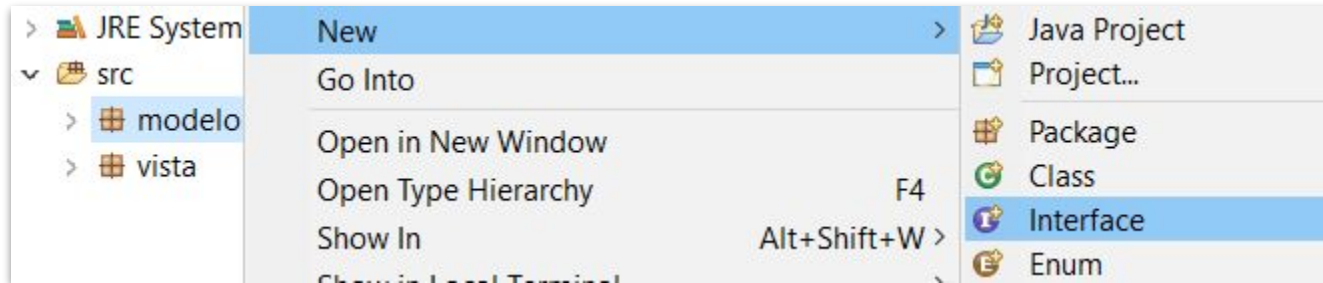


Interfaces

- Como las interfaces definen un comportamiento, su **nombre** suele utilizar los **sufijos**:
 - -able Clonable, Serializable, Ejecutable, Arrancable
 - -or Servidor, Buscador
 - -ente Urgente, Cliente

Interfaces. Declaración

- Para **crear** una Interfaz usamos el menú de Eclipse
- Las interfaces son parte del **modelo**



Interfaces. Declaración

- Declaración de una Interfaz

```
public interface MiInterfaz {  
    //Atributos. Si existen, siempre constantes (final)  
    public final int CONSTANTE=1;  
    //Métodos. Todos abstractos (sin código)  
    public abstract void metodo();  
}
```

- Si la interfaz no se declara **public**, sólo será visible desde el paquete
- No es obligatorio el uso de la palabra reservada **abstract** en la cabecera de los métodos aunque sí es recomendable (por claridad del código)

Interfaces. Declaración

- Se utiliza la palabra reservada `interface` en lugar de `class`
- Todos los miembros de la interfaz (atributos y métodos) son `public` de manera implícita.
 - No es necesario indicar el modificador `public`, aunque puede hacerse
- Todos los atributos son de tipo `final` y `public` (tampoco es necesario especificarlo), es decir, constantes y públicos.
 - Hay que darles un valor inicial
- Todos los métodos son abstractos también de manera implícita (tampoco hay que indicarlo).
 - No tienen cuerpo, tan solo la cabecera

Interfaces. Declaración

- Ejemplo de la Interfaz Arrancable:

```
public interface Arrancable {  
    public abstract void arrancar();  
}
```

Interfaces. Implementación

- Una vez declarada una Interfaz, cualquier Clase puede **implementarla** utilizando la palabra reservada **implements**

```
public class Coche implements Arrancable{
```

- Ahora Coche **está obligada** a escribir los métodos de la Interfaz Arrancable
- Si no lo hace, el compilador dará un error

```
public class Coche implements Arrancable{  
    @Override  
    public void arrancar() {  
        //Código para arrancar un Coche  
    }  
}
```

Interfaces. Implementación

- Una clase puede tener **varios comportamientos**
- Es decir, **implementar varias Interfaces**

```
public class Coche implements Arrancable, Imprimible{
```

Interfaces

EJERCICIO

- [Ficheros .java](#)
- Crea la Interfaz “Sancionable” que sólo contiene el método “sancionar”
- Alumnos y Profesores pueden ser sancionables por tanto, ¿quién implementará la interfaz (Persona, Alumno y/o Profesor)?
- Cuando se sancione a un profesor se le asignará un sueldo de 1000€
- Un alumno se sanciona y su nota media pasa a ser de 5 puntos

Interfaces

EJERCICIO

[Solución](#)

Herencia de interfaces

- Las interfaces pueden **heredar** unas de otras
 - Incluso puede heredar de más de una interfaz
- Cuando lo hacen heredan **todos los atributos y métodos** públicos

```
public interface ArrancableElectrico extends Arrancable{
```

- La clase que implemente la interfaz ArrancableElectrico debe implementar también los métodos abstractos de Arrancable

Variables de tipo interfaz

- También se pueden **declarar variables** cuyo tipo es una interfaz

```
Arrancable vehiculoArrancable;
```

- Y con esa variable podemos **crear cualquier tipo de objeto** que implemente la interfaz

```
Arrancable vehiculoArrancable=new Coche();  
vehiculoArrancable.arrancar();
```

- Pero tenemos la misma limitación que con el polimorfismo:
`vehiculoArrancable` **sólo puede ejecutar** los métodos de la Interfaz `Arrancable` (y los que haya heredado la interfaz)

Variables de tipo interfaz

EJERCICIO

- Accede a la documentación oficial del Java y revisa la interfaz Set
- En Eclipse declara un conjunto de tipo Set e instáncialo como un HashSet
- Ahora declara otro conjunto de tipo HashSet e instáncialo como un HashSet
- ¿Los dos objetos pueden ejecutar exactamente los mismos métodos? ¿Por qué?

Clases anónimas

- Lo habitual es que las interfaces sean **implementadas por las clases**
- Pero hay veces que necesitaremos usar una interfaz en un momento **puntual**
- Para eso crearemos sobre la marcha una clase sin nombre, **anónima**, que implementará los métodos que nos interesen de la interfaz

Clases anónimas

- Ejemplo

- Nos encontramos con un vehículo que no podemos definir bien (ni es un coche, ni una moto) pero que queremos arrancar
- Crearemos un objeto de la interfaz y para crearlo, utilizamos como constructor el nombre de la interfaz (aunque no existe un constructor como tal)

```
Arrancable vehiculoRaro = new Arrancable() {  
    @Override  
    public void arrancar() {  
        System.out.println("Vehículo raro arrancado");  
    }  
};  
vehiculoRaro.arrancar();
```

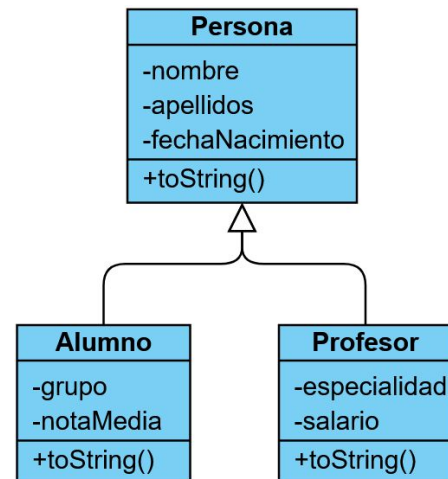
Interfaz Comparable

- Cuando queramos establecer un **criterio de comparación** entre dos objetos de una clase tendremos que implementar la interfaz `Comparable`
- La interfaz obliga a implementar el método `compareTo(o)` que devuelve
 - `<0` si this va antes que o
 - `>0` si this va después que o
 - `==0` si son iguales
- En la Unidad 6 ya trabajamos con la interfaz `Comparable`

Interfaz Comparable

EJERCICIO

- Partiendo de la lista polimórfica de Alumnos y Profesores, establece como criterio de ordenación “alfabético por apellidos”
- Ordena la lista
- Ordena la lista en orden inverso



Interfaz Comparator

- Comparable nos da un **criterio general (orden natural)** de ordenación que van a usar otros métodos de la API como `sort()`
- Pero es habitual que en una aplicación queramos establecer **diferentes criterios de ordenación**
 - Alfabético por nombre, por edad, fecha de alta, en sentido ascendente o descendente...
- Para eso tenemos la interfaz Comparator que tiene un único método:

```
int compare(Object ob1, Object ob2)
```

- Devuelve
 - `<0` si ob1 va antes que ob2
 - `>0` si ob1 va después que ob2
 - `==0` si son iguales

Interfaz Comparator

- Comparator **no se va a implementar** por ninguna clase que queramos ordenar
- Se va a implementar sobre un objeto al que vamos a llamar ***comparador***
- En realidad tampoco crearemos un objeto, si no que se implementará **sobre una clase anónima** cuando necesitemos usar la ordenación

Interfaz Comparator

EJEMPLO

- En la estructura de herencia de Persona, Alumno y Profesor tenemos una lista que queremos ordenar
- Si usamos el método sort y le pasamos como parámetro el conjunto, hará la ordenación natural según se especifica en compareTo()
- Pero vemos que el método sort está sobrecargado y admite un objeto **Comparator** como segundo parámetro

```
Collections.sort
```

```
sort(List<T> list) : void - java.util.Collections
```

```
sort(List<T> list, Comparator<? super T> c) : void - java.util.Collections
```

Interfaz Comparator

Collections.sort

• **sort**(List<T> list) : void - java.util.Collections

• **sort**(List<T> list, Comparator<? super T> c) : void - java.util.Collections

- Es ahí donde vamos a crear un objeto anónimo de tipo Comparator

```
Collections.sort(listaPersonas, new Comparator<Persona>() {  
    @Override  
    public int compare(Persona pers1, Persona pers2) {  
        //Ordenación alfabética por nombre  
        return pers1.getNombre().compareTo(pers2.getNombre());  
    }  
});
```

Interfaz Comparator

Objeto de tipo Comparator creado como clase anónima

```
Collections.sort(listaPersonas, new Comparator<Persona>() {  
    @Override  
    public int compare(Persona pers1, Persona pers2) {  
        //Ordenación alfabética por nombre  
        return pers1.getNombre().compareTo(pers2.getNombre());  
    }  
});
```

Interfaz Comparator. Ordenación inversa

- Para hacer una **ordenación inversa** se puede definir en la implementación del método `compare()`

```
Collections.sort(listaPersonas, new Comparator<Persona>() {  
    @Override  
    public int compare(Persona pers1, Persona pers2) {  
        //Ordenación alfabética por nombre descendente  
        return -(pers1.getNombre().compareTo(pers2.getNombre()));  
    }  
});
```

Interfaz Comparator. Ordenación inversa

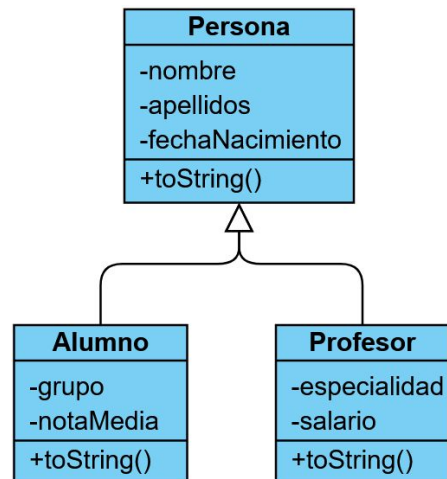
- O usar el método `reversed()` de `Comparator`

```
Collections.sort(listaPersonas, new Comparator<Persona>() {  
    @Override  
    public int compare(Persona pers1, Persona pers2) {  
        //Ordenación alfabética por nombre descendente  
        return (pers1.getNombre().compareTo(pers2.getNombre()));  
    }  
}).reversed());
```

Interfaz Comparator

EJERCICIO

- Ordena la lista por fecha de nacimiento en orden de más reciente a más antigua
- Prueba a invertir el orden anterior
 - Cambiando la definición del método compare()
 - Usando el método reversed() de Comparator



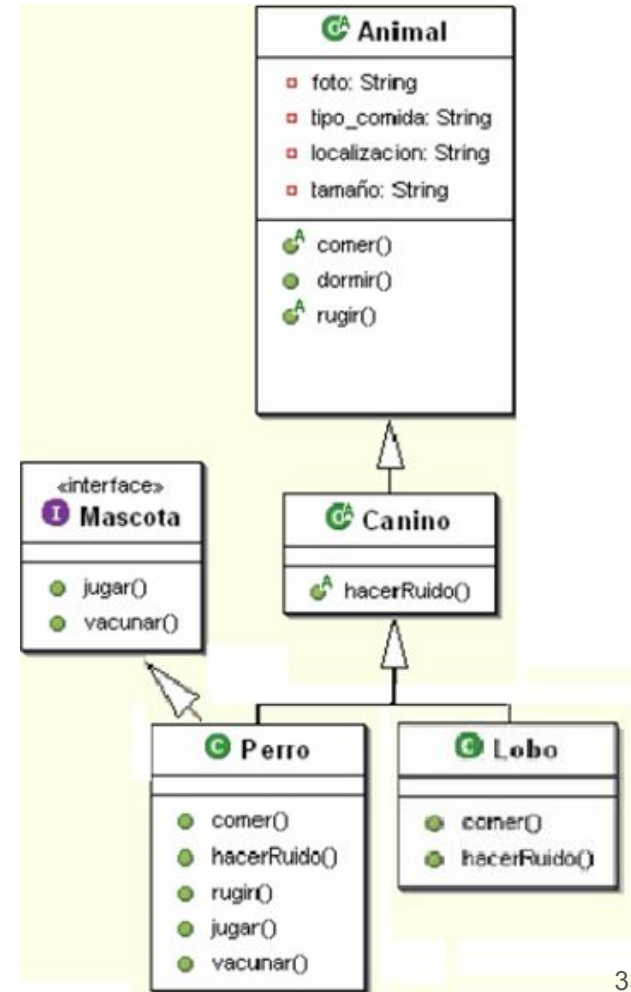
Interfaces funcionales

- Una interfaz funcional es aquella que sólo tiene **un único método abstracto**
- Su uso está relacionado con las [expresiones lambda](#) de Java 8 que quedan fuera de este curso

¿Interfaces o herencia?

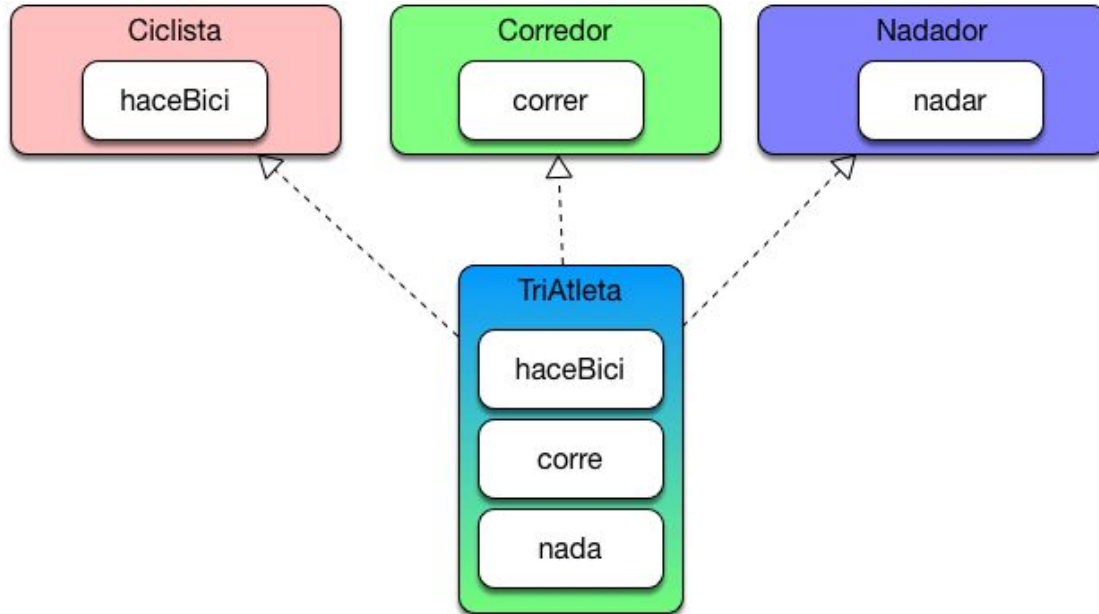
¿Cuándo usar uno u otro?

- Usamos herencia cuando haya una relación clara entre las clases
- Usamos interfaces cuando queremos añadir un comportamiento extra o específico a alguna de las clases



¿Interfaces o herencia?

- También se pueden ver las interfaces como una forma de hacer **herencia múltiple**



FIN!